

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Patrón repositorio



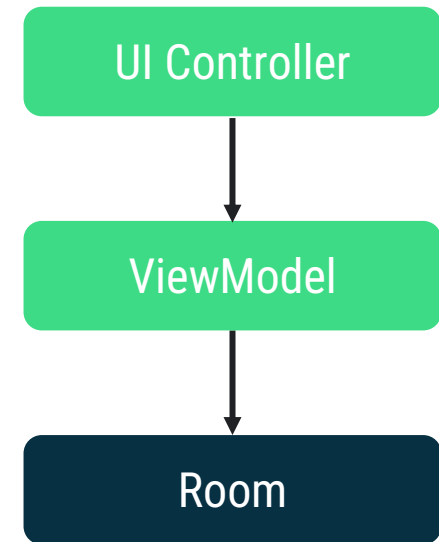
Universidad
Rey Juan Carlos

App con múltiples fuentes de datos

En las aplicaciones que hemos construido hasta ahora, nuestros objetos del modelo de vista contenían referencias directas a objetos DAO para consultar la base de datos local.

Para la mayoría de aplicaciones, donde habitualmente se necesita obtener datos de recursos externos (por ejemplo, la red), se hace muy difícil escalar la aplicación con la arquitectura existente.

La lógica que tendríamos que añadir al modelo de vista sería bastante compleja, y **no queremos que el ViewModel sea el responsable de gestionar la recuperación de datos.**



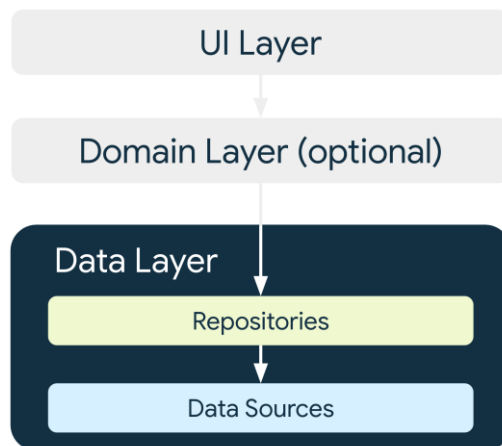
Patrón repositorio

Patrón repositorio

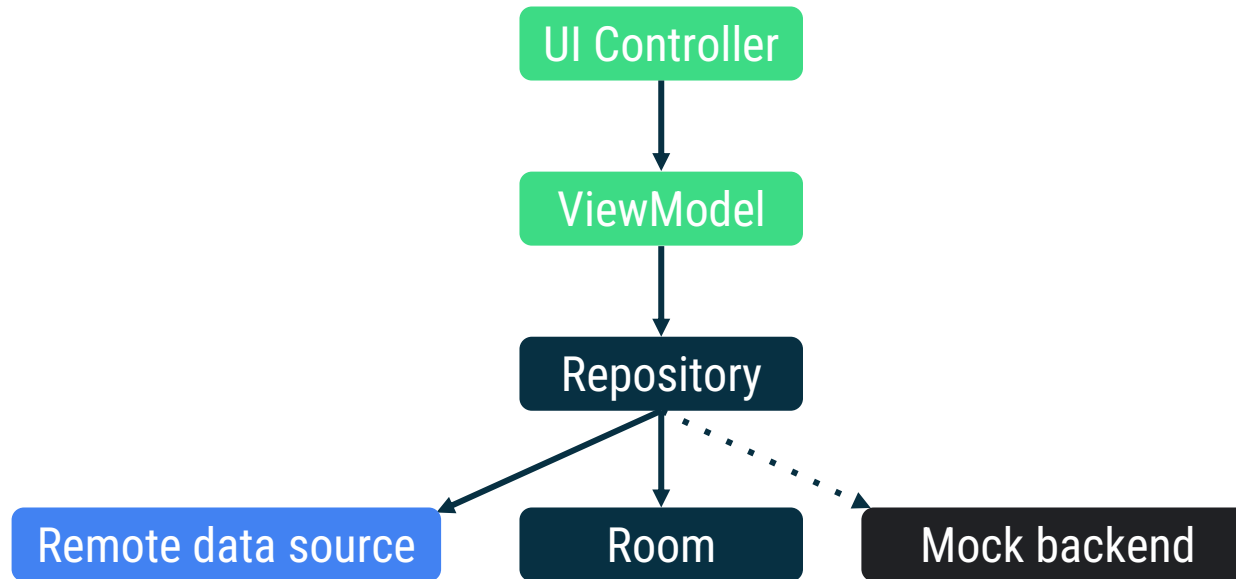
El patrón de repositorio en Android es un enfoque de diseño que proporciona una forma estructurada y organizada de **gestionar el acceso y la manipulación de datos**.

Sirve como **intermediario entre diferentes fuentes de datos** (como bases de datos, servicios de red) **y el resto de la aplicación**.

El objetivo principal es **abstraer los detalles de acceso a los datos**, promoviendo una arquitectura limpia, la reutilización del código y la facilidad de mantenimiento.



Arquitectura de la app usando Repositorio



- Puede abstraer múltiples fuentes de datos, separándolo de la capa de UI
- Permite una recuperación rápida utilizando la base de datos local mientras se envía una solicitud de red para actualizar los datos (más lento)
- Permite probar las fuentes de datos de manera independiente

Implementación de un repositorio

Cuando se trata de implementar el patrón de repositorio en tu aplicación, **no hay requisitos explícitos** (en términos de una clase que debas extender o métodos que debas declarar). Pero...

debe proporcionar una interfaz común para acceder a los datos:

- Exponer funciones para consultar y modificar los datos subyacentes

dependiendo de tus fuentes de datos, el repositorio puede:

- Mantener una referencia a la DAO, si sus datos están en una base de datos
- Realizar peticiones de red, si se conecta a un servicio web

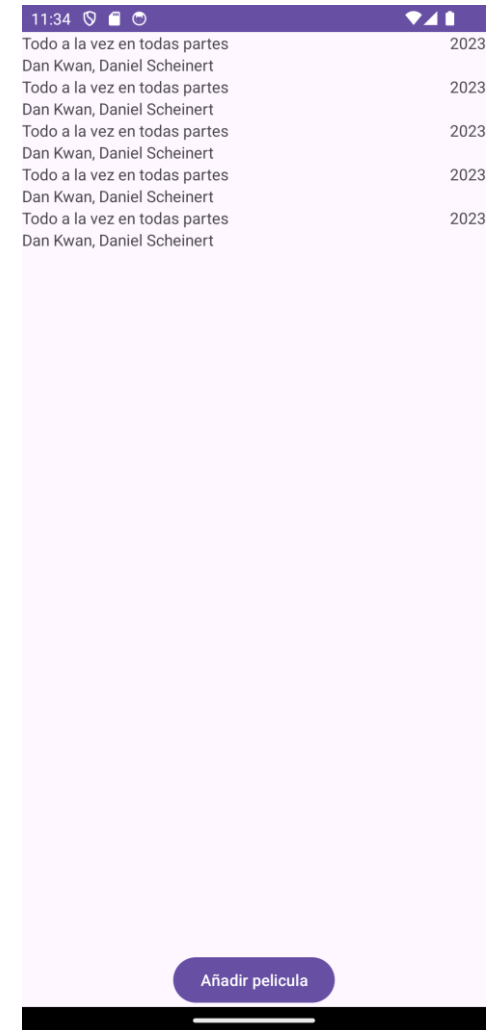
Repositorio simple con Room y API Rest

```
class LibrosRepository(LibrosRepository.kt  
    private val dao: LibrosDAO, // DAO para interactuar con bdd local (Room)  
    private val api: LibrosAPI // API para hacer peticiones a nuestro servidor remoto (Retrofit)  
) {  
    var libros: LiveData<List<Libro>> = dao.getAll()  
    fun getAllLibros(): LiveData<List<Libro>> {  
        if (libros.value?.isEmpty() == true) {  
            var remoteLibros = getLibrosRemote()  
            // TODO Insertar películas remotas en bdd local  
        }  
        return libros  
    }  
  
    // Función para recuperar los datos de películas de un servidor remoto con la API Rest  
    fun getLibrosRemote(): List<Libro> = TODO()  
  
    fun addLibro(libro: Libro) {  
        dao.insert(libro)  
    }  
}
```

Ejercicio 1

Siguiendo con la aplicación de la sesión anterior, donde teníamos una actividad que mostraba las películas almacenadas en una base de datos Room accesible desde un ViewModel:

- Implementa una clase repositorio para que se encargue de las tareas de recuperación de datos de la base de datos Room
- Adapta las clases necesarias en la aplicación para que haga uso del patrón repositorio



Separación modelo de datos-dominio

Además de abstraer las diferentes fuentes de datos de la aplicación y separarlas de la capa de UI (ViewModel), **el repositorio debe emplearse para separar el modelo de datos** (lo almacenado en Room o en remoto) **del modelo de dominio** (datos que usa nuestra aplicación).

El repositorio es responsable de:

- Exponer únicamente los datos de dominio (los utilizados por la app, independientemente de la información almacenada en las diferentes fuentes de datos).
- Interactuar con las fuentes de datos utilizando los objetos adecuados que representen la información almacenada (p. ej. Entity).

Clase dominio vs de fuente de datos

Las entidades definen las columnas de la tabla en la base de datos Room, por lo que sus atributos solo pueden tomar tipos básicos (int, float, string...)

```
@Entity(tableName = "libros")
data class LibroEntity(// Antes llamada Libro !!!
    @PrimaryKey(autoGenerate = true) val id: Int? = null,
    val titulo: String,
    val fecha : String // ejemplo: 2018-03-27
)
```

LibroEntity.kt

Para la lógica interna de nuestra aplicación, eso nos puede suponer una limitación. Por ello, es conveniente diferenciar las clases de dominio de las clases de entidad (Room) o API (Retrofit).

```
data class Libro(
    val id: Int?,
    val titulo: String,
    val fecha: LocalDate
)
```

Libro.kt

Maapeo entre clases

Al separar las clases de datos de dominio de las de fuentes de datos, es necesario implementar funciones para las conversiones entre las mismas.

```
fun LibroEntity.toDomain(): Libro {  
    return Libro(  
        id = this.id,  
        titulo = this.titulo,  
        fecha = LocalDate.parse(this.fecha)  
    )  
}
```

LibroMapper.kt

```
fun Libro.toEntity(): LibroEntity {  
    return LibroEntity(  
        id = this.id,  
        titulo = this.titulo,  
        fecha = this.fecha.toString()  
    )  
}
```

Repositorio como mediador

El repositorio, por tanto, se encarga de **recuperar los datos de la fuente** que sea y **mostrarlos con el formato de datos de dominio** esperado por la aplicación, realizando las transformaciones necesarias.

Para ello, podemos utilizar la función *map*, que permite realizar cambios dentro de un *LiveData* o de una *List*, juntos con las funciones de mapeo implementadas.



```
class LibrosRepository(private val dao: LibroDAO) {  
    fun getAllLibros(): LiveData<List<Libro>> = dao.getAll().map { listaLibros ->  
        listaLibros.map { it.toDomain() } }  
  
    // Resto de funciones expuestas (p. ej.: addLibro, deleteLibro, etc)  
}
```

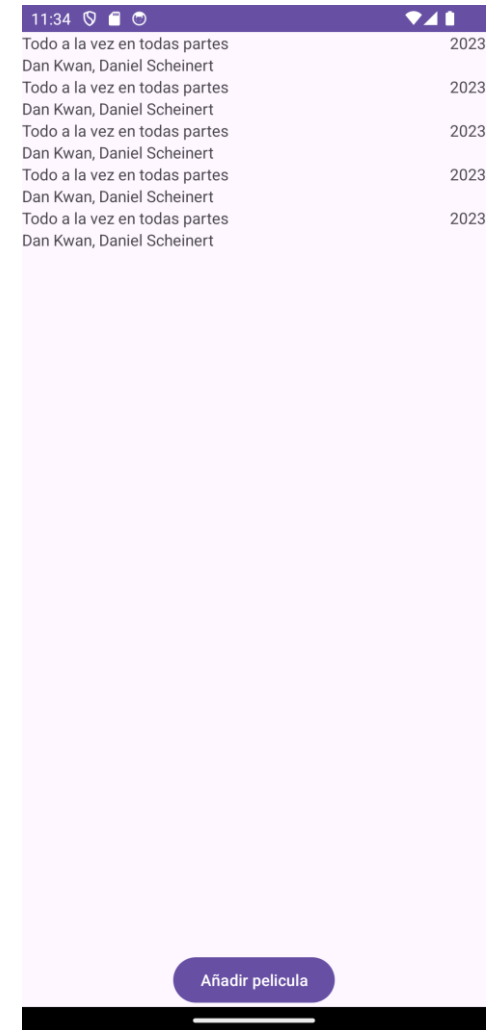
LibrosRepository.kt

Ejercicio 2

Siguiendo con el ejercicio anterior:

- Modifica la BBDD para que el campo *año* sea la fecha de *estreno*, dejando de ser un entero y almacenándose como una String “YYYY-MM-DD”
- Implementa una clase de dominio para la Pelicula, donde el atributo *estreno* sea del tipo [LocalDate](#)
- Implementa las funciones de conversión necesarias entre la entidad y la clase de dominio

Importante: en el RecyclerView la visualización no cambia. Es decir, se seguirá mostrando únicamente el año de estreno de la película, no la fecha completa.



Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>